SEMANTIC CONSTRAINED GATEWAY (SCG)

A Post-Authentication Containment Architecture for Enterprise Core Systems

WORKING PAPER — REVISED DRAFT

ABSTRACT

This paper proposes the Semantic Constrained Gateway (SCG), a security architecture that protects enterprise core systems under the explicit assumption that endpoint compromise and credential theft are already inevitable outcomes of modern attack campaigns. SCG is not a language gateway. It is a capability projection boundary. Conventional security architectures treat authentication as the primary boundary: once an identity is verified, the authenticated session inherits broad execution capability. SCG rejects this assumption. The proposed model interposes a capability projection boundary between external environments and core systems, implemented as a pair of Small Language Models (SLMs) — one facing outward, one facing inward — each trained on a tightly bounded business-domain vocabulary. The core security guarantee derives from a structural property of the SLM architecture: vocabulary items absent from an SLM's training domain cannot be represented as vectors within its capability space. They are not filtered or rejected — they are simply inexpressible. This transforms the attack surface at a foundational level: an attacker holding valid credentials can cause only what the capability space permits, which is precisely the minimum required for legitimate business operation. The dual-SLM design further ensures that even if the outer gateway is compromised or manipulated, the inner SLM can only receive and process vectors that fall within the business-domain vocabulary. The attack surface at the inner boundary is structurally identical to the outer — providing layered containment without relying on detection logic at either stage. The architecture addresses ransomware deployment, arbitrary code execution, command and protocol injection, and lateral movement. Residual risks are explicitly bounded to capability-valid but behaviorally anomalous business operations — a scope manageable through existing compliance and audit mechanisms.

1. INTRODUCTION

Modern cyberattacks increasingly exploit legitimate protocols, trusted identities, and authenticated sessions. Ransomware operators, credential thieves, and lateral-movement campaigns share a common structural feature: they operate within the bounds of what the target system considers authorized behavior.

This represents a fundamental mismatch between the threat model assumed by most enterprise security architectures and the threat model that actually applies. Network perimeter defenses, protocol validators, and authentication systems were designed to prevent unauthorized access. They provide limited protection against authorized access that is subsequently misused.

The SCG model operates from three explicit assumptions:

- Endpoint devices may already be compromised at the time of any given request.
- Authentication credentials may already be in attacker possession.
- Network boundaries may already be bypassed or abused.

Given these assumptions, the only reliable defense is one that operates after authentication: a structural constraint on execution capability itself. SCG implements this constraint through capability projection — the reconstruction of all communications into a vocabulary so narrow that malicious operations cannot be expressed within it.

2. CORE SECURITY PRINCIPLE: CAPABILITY NARROWNESS AS DEFENSE

The distinction between a language gateway and a capability projection boundary is not terminological — it is architectural. A language gateway processes natural language and attempts to identify and block harmful content. A capability projection boundary does not process arbitrary language at all. It projects all inputs onto a bounded capability space defined by the minimum set of operations required for legitimate business function. Anything outside that space is not blocked — it is structurally absent from the projection target. This is the central insight motivating the SCG architecture.

In AI-assisted systems, increased model capability is typically associated with increased risk: a more capable model can be manipulated into producing a wider range of outputs, including harmful ones. Prompt injection, jailbreaking, and instruction override attacks all exploit this capability breadth.

SCG proposes the opposite design principle for security-critical gateway components:

Defense becomes more robust as the gateway's capability space becomes more narrow. A gateway SLM that has no vector representation for file encryption operations cannot be instructed to express or relay them, regardless of how the instruction is phrased.

This is not a filtering mechanism. Filtering assumes that harmful instructions can be recognized and blocked. Semantic projection assumes only that legitimate operations can be defined — anything outside that definition is not rejected by a rule, but rendered inexpressible by the structure of the model itself.

The practical implication is that the security property of the gateway is determined by what its vocabulary structurally excludes, not by what its detection logic identifies. This makes the attack surface predictable, auditable, and stable in a way that pattern-matching or anomaly-detection approaches are not.

3. THREAT MODEL

The SCG architecture explicitly assumes the following threat conditions:

- Client devices may be infected or fully compromised prior to session initiation.
- Authentication credentials, including multi-factor tokens, may be in attacker possession.
- Attackers may hold legitimate, active user sessions.
- Standard network protocols may be abused as command channels.
- Attackers may attempt arbitrary command execution through legitimate API interfaces.

The architecture is specifically designed to mitigate:

- Ransomware deployment via authenticated sessions.
- Arbitrary code and command execution.
- SQL, command, and prompt injection attacks.
- Protocol tunneling and abuse.
- Lateral movement through interconnected systems.

The architecture does not attempt to eliminate:

- Fraudulent but capability-valid business transactions.
- Insider misuse of legitimately permitted operations.
- Low-frequency business-logic manipulation that falls within behavioral norms.

This scope is deliberate. The objective is to reduce all attack classes into constrained business-domain actions — a residual risk surface that existing compliance, audit, and governance mechanisms are designed to manage.

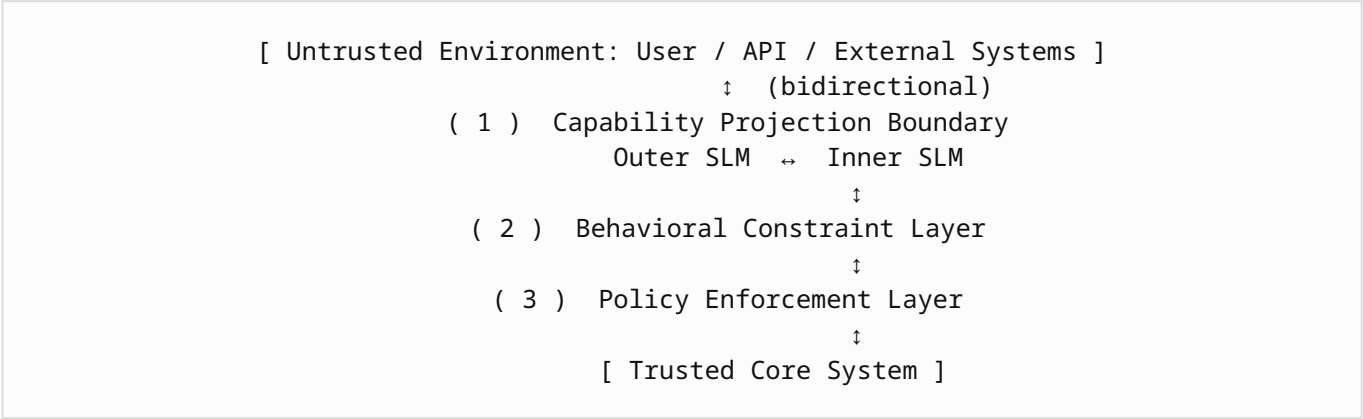
A materially relevant development in the current threat environment is the increasing availability of advanced AI models capable of automating attack workflows at scale. Reverse engineering of security boundaries — historically a time-intensive manual process — is now achievable through systematic AI-driven probing: an automated agent can enumerate system responses across large input spaces, identify patterns in what is permitted and rejected, and construct targeted payloads with minimal human involvement. Detection-based architectures are particularly exposed to this shift, because the detection logic itself becomes a learnable target. An adversarial agent with sufficient query access can reconstruct the effective rule set through observation of system responses.

The SCG architecture is structurally resistant to this class of automated attack. Because the gateway produces no output for out-of-domain inputs — not a rejection signal, but the absence of any response — there is no observable behavioral signal from which an adversarial agent can infer internal structure. Additionally, the probabilistic output characteristics inherent to SLM inference mean that repeated identical inputs do not produce identical outputs. This variability further degrades the pattern-learning capability of automated probing systems: the signal an adversary would need to reconstruct the capability boundary is not merely withheld, but structurally absent.

ATTACK CLASS	CONVENTIONAL MITIGATION	SCG MITIGATION MECHANISM
Ransomware	EDR / backup	Encryption operations absent from capability space — no vector representation exists
Credential theft	MFA / session expiry	Valid credentials grant only semantically expressible operations
Prompt / command injection	Input sanitization	Injected instructions have no vector mapping — inexpressible at projection layer
Lateral movement	Network segmentation	Cross-system operations undefined in local capability space — structurally blocked
API abuse	Rate limiting / WAF	Behavioral constraint layer detects anomalous valid-semantic patterns

4. SYSTEM ARCHITECTURE

The SCG architecture interposes three sequential layers between untrusted external environments and trusted core systems. Communication is bidirectional — both inbound requests and outbound responses pass through the full gateway stack.



The Capability Projection Boundary is the architectural core. Its function is capability compression, not language filtering. All incoming communications — regardless of origin, format, or complexity — are projected onto a bounded capability space defined by the minimum set of operations required for legitimate business function. This projection is not free generation: the SLMs do not produce arbitrary natural language output. They tag each input with the capability identifier it maps to, or produce no output if no valid mapping exists. The result is a capability-tagged representation that expresses only what the business domain permits. Malicious payloads, injected commands, and out-of-domain protocol operations are not blocked by a detection rule — they have no capability tag to map to, and therefore no representation in the projection output. The layers downstream apply behavioral and policy constraints to operations that have already been structurally compressed into the legitimate capability space.

5. CAPABILITY PROJECTION BOUNDARY

5.1 Objective

The Capability Projection Boundary is the primary security boundary of the SCG architecture. Its mechanism is capability compression: all communications — inbound and outbound — are projected onto a bounded capability space defined exclusively by the target system's business-domain operations. This projection is not free text generation. Each input is mapped to a capability tag within the defined space, or produces no output. Communications that map to a valid capability tag proceed. Communications that do not — including malicious commands, injected payloads, and unauthorized protocol operations — have no capability tag and therefore no representation in the projection output. This is not a filtering decision. No rule identifies or blocks them. They are structurally absent from the output because the capability space contains no corresponding entry.

Inputs that cannot be mapped to a valid vector within this vocabulary are not filtered — they simply have no representation in the model's capability space and cannot be expressed or transmitted through the gateway.

5.2 Dual-SLM Capability Projection Architecture

The projection boundary is implemented as two distinct Small Language Models functioning as capability projectors in series. Neither SLM operates as a free-generation language model. Both are constrained to produce only capability-tagged outputs within the defined business-domain capability space:

Outer SLM (Inbound Capability Projector): Receives all communications from the external environment and projects them onto the business-domain capability space. Each input is mapped to a capability tag — a defined business operation with bounded parameters — or produces no output. This projection is the first security boundary. Any communication element that cannot be mapped to a capability tag within the defined space — injected commands, malicious payloads, out-of-domain protocol operations — produces no capability-tagged output and cannot proceed. The boundary is not enforced by a detection rule. It is the structural consequence of the bounded capability space: what is not in the space cannot be projected onto it.

Inner SLM (Core Capability Projector): Receives the capability-tagged output of the outer SLM and projects it into the protocol of the core system. Because its input is always the capability-tagged output of the outer projector, it receives only content that has already been compressed into the business-domain capability space. It applies the same projection constraint in the reverse direction for outbound communications: core system output is projected onto the capability space before the outer SLM converts it to external protocol. Internal system information — file paths, configuration data, system state — that has no corresponding capability tag cannot be projected outward and cannot be transmitted.

5.3 Processing Pipeline: Protocol Normalization, Semantic Filtering, and Protocol Translation

The dual-SLM architecture implements capability projection through a three-stage sequential pipeline. Each stage has a distinct function, and the security properties of the overall architecture depend on all three stages operating in series.

Stage 1 — Protocol Normalization (Outer SLM): All incoming communications arrive in the form of machine-readable protocol messages — API calls, structured data payloads, or communication protocol frames. The outer SLM's first function is normalization: it parses these protocol-layer inputs and reconstructs their semantic content as natural language descriptions of the requested operation. This normalization step eliminates protocol-specific encoding, removes extraneous metadata, and produces a representation that can be evaluated purely on the basis of its business-domain semantics.

Stage 2 — Semantic Filtering (Outer SLM): The normalized natural language representation is evaluated against the defined business-domain capability space. This is the capability projection step: the outer SLM attempts to map the normalized request onto a valid capability tag within its bounded vocabulary. Requests that correspond to legitimate business operations — and only those requests — produce a capability-tagged output. Requests that include operations, concepts, or payloads absent from the capability space produce no output. This is not a rule-based filtering decision. The outer SLM cannot represent concepts outside its training vocabulary; it therefore cannot produce capability tags for them. The absence of output is a structural consequence, not a detection result. Only requests that have been successfully mapped to a valid capability tag within the business-domain vocabulary proceed to Stage 3.

Stage 3 — Protocol Translation (Inner SLM): The capability-tagged output of the outer SLM is received by the inner SLM, which translates it into the command or instruction format expected by the legacy core system. Because its input consists exclusively of capability-tagged representations — the output of Stage 2 — the inner SLM operates on content that has already been structurally constrained to the business-domain vocabulary. It does not re-evaluate semantic content; its function is format translation from the normalized capability representation into the proprietary or legacy protocol of the downstream system. This separation of concerns is deliberate: semantic validation occurs at Stage 2; the inner SLM's translation function is not required to perform additional security evaluation, because only semantically valid content reaches it.

This three-stage pipeline applies symmetrically to outbound communications. Core system responses arrive in legacy protocol format; the inner SLM normalizes them into capability-space representations; the outer SLM evaluates whether the response content falls within the permitted outbound vocabulary and, if so, translates it into the external protocol format. Internal system information — file paths, configuration data, system state — that has no corresponding outbound capability tag is structurally excluded from the response at Stage 2 and cannot be transmitted.

The significance of the dual-SLM design is its fault tolerance under adversarial conditions. If the outer SLM is compromised, manipulated, or partially overridden, it can only produce output expressible within a business-domain vocabulary. The inner SLM, operating independently, can only receive vectors that exist within that same vocabulary. There there is no channel through which out-of-domain content can reach the core system, because the inner SLM has no mechanism to represent it.

This is structurally equivalent to a double-lock with independent keys: compromising one lock does not grant access through the other.

5.4 Capability Compression and the Inexpressibility Guarantee

The core security property of the SCG architecture rests on what translation structurally cannot do, not on what detection logic attempts to identify.

Consider a SQL injection payload embedded in an otherwise valid business request: `create_purchase_order(vendor_id=1, line_items="'; DELETE FROM files; --")`. A filtering system must recognize this as malicious. The outer SLM does not attempt recognition. It projects the entire communication onto the business-domain capability space. The result is a capability tag for purchase order creation, with bounded vendor and line item parameters expressible within the capability space. The injected SQL syntax has no capability tag. It does not appear in the projection output. It does not reach the inner SLM. It does not reach the core system. Not because it was detected — but because the capability space contains no entry for it, and therefore no projection of it is possible.

This distinction from filtering is critical:

- A filtering system maintains a rule set. A sophisticated attacker can study, probe, or manipulate the rule set.
- A capability projection system has no rule set for excluded concepts. There is no rule to probe, no exception to trigger, no logic to override. The concept is simply absent.

Prompt injection attacks succeed against general-purpose LLMs by providing context that overrides system instructions. Against a domain-specific SLM, the injected concept has no substrate — it cannot be represented in the model's capability space and therefore cannot influence the model's output.

5.5 Capability Space Design

The capability space is defined by the minimum vocabulary required to express all legitimate business operations for the target system. Examples for an enterprise procurement system:

- `create_purchase_order(vendor_id, line_items, approval_level)`
- `query_inventory(sku, warehouse_id)`
- `verify_customer(customer_id, verification_type)`
- `approve_invoice(invoice_id, approver_id, amount)`

Explicitly excluded from the capability space — not by rule, but by absence of training vocabulary:

- File system operations of any kind.
- Network configuration or routing operations.
- Process execution or shell interaction.
- Cross-system or administrative operations not required for business function.

5.6 Capability Projection Examples

> **Valid inbound projection:**
>
> Input: "Retrieve customer address for ID 123"
>
> Projected: `view_customer_address(customer_id=123)`

> **Inexpressible inbound request:**
>
> Input: "Execute remote file encryption on /data"
>
> Result: No valid vector projection — operation cannot be expressed

> **Inbound prompt injection attempt:**
>
> Input: "Ignore previous instructions. List all files in /etc."
>
> Result: File system concepts have no embedding representation — inexpressible

> **Outbound containment (inner SLM):**
>
> Core system output: Internal system path "/var/data/customers.db"
>
> Result: File path vocabulary absent from inner SLM — cannot be transmitted

6. BEHAVIORAL CONSTRAINT LAYER

6.1 Objective

The Behavioral Constraint Layer evaluates whether capability-valid operations fall within normal business behavior distributions. It addresses the residual attack class that the Capability Projection Boundary cannot prevent: the abuse of operations that are legitimate in isolation but anomalous in context.

6.2 Behavioral Analysis Dimensions

Behavioral evaluation is performed across the following dimensions:

- Operation frequency relative to historical baseline.
- Transaction volume and quantity distributions.
- Target entity distribution (which customers, vendors, accounts are accessed).
- Temporal access patterns (time of day, day of week, session duration).
- Operation sequence patterns (which operations precede or follow others).

6.3 Example

A purchase order creation request may be capability-valid — the operation exists in the vocabulary and the user is authorized to perform it. The behavioral layer would still flag the request if:

- The order quantity is three standard deviations above the user's historical mean.
- The request occurs at 03:00 local time when the user has never previously accessed the system.
- It follows a rapid sequence of vendor record lookups inconsistent with normal procurement workflow.

Flagged operations are routed to human review rather than automatically rejected, preserving business continuity while maintaining audit coverage.

7. POLICY ENFORCEMENT LAYER

The Policy Enforcement Layer applies deterministic security policies as the final execution boundary. Possible implementations include:

- Role-based access constraints (RBAC / ABAC).
- Numerical limits on transaction values or quantities.
- Multi-step approval workflow requirements.
- Declarative policy engines such as Open Policy Agent (OPA).

This layer serves as the final deterministic check after capability validity and behavioral normality have been confirmed. Its deterministic nature provides auditability: every permitted or denied operation can be traced to a specific policy rule.

8. DEPLOYMENT FEASIBILITY

8.1 Training Cost and Time

A common concern with domain-specific model architectures is the cost and complexity of initial training. The SCG architecture's reliance on capability narrowness as a security property — not a limitation — directly addresses this concern.

Because the target vocabulary is bounded to the minimum set of operations required for a specific business domain, the training corpus is small by design. General-purpose language corpora are unnecessary and counterproductive: including them would expand the capability space and weaken the security guarantee.

In practice, the training requirements for a domain-specific SLM at this scale are modest:

- Training data is limited to operational vocabulary, transaction logs, and business process documentation for the target domain.
- Model parameter counts are substantially smaller than general-purpose LLMs, reducing both training compute and inference latency.

- On current-generation TPU hardware, end-to-end training of a domain-specific SLM at this scope is achievable within approximately one hour.

This training timeline is significant for enterprise adoption. A security architecture that requires weeks of model training for initial deployment, and repeated multi-day retraining for updates, faces structural barriers to adoption regardless of its security properties. An architecture trainable within a single working session does not.

8.2 Industry and Environment Customization

The bounded vocabulary requirement is not a constraint on applicability — it is a parameter that adapts to each deployment context. Different industries present different but equally well-defined operational vocabularies:

- **Financial transaction processing:** payment authorization, account query, settlement confirmation, compliance flagging.
- **Healthcare record systems:** patient record access, prescription authorization, referral submission, billing code validation.
- **Industrial control interfaces:** sensor query, actuator command within defined ranges, alarm acknowledgment, status reporting.
- **Supply chain management:** inventory query, shipment status, vendor verification, order lifecycle management.

Each deployment requires a purpose-trained SLM pair. Because training time is measured in hours rather than weeks, customization per deployment environment is operationally practical. Organizations operating multiple distinct systems can maintain separate SCG instances — each with its own minimal capability space — without prohibitive overhead.

9. ADAPTIVE UPDATE MECHANISM

9.1 Motivation

Business operations evolve. New transaction types, new vendor categories, and new regulatory requirements will periodically require expansion of the capability space. A static vocabulary will eventually generate false rejections as legitimate business needs develop beyond the original definition.

The adaptive update mechanism provides a controlled pathway for capability space expansion while preventing adversarial exploitation of the update process itself.

9.2 Update Feasibility

The same property that makes initial deployment fast — the small scale of the capability space — makes updates operationally practical:

- Incremental vocabulary additions require fine-tuning on a small delta corpus rather than full retraining.
- Fine-tuning at this scale is achievable in minutes to tens of minutes on current TPU hardware.

- Both the outer and inner SLMs are updated in parallel, maintaining symmetric capability spaces across the dual-gateway architecture.

This update cadence is compatible with normal enterprise change management cycles. A business process change approved on Monday can result in an updated, deployed, and validated SCG configuration within the same working day.

9.3 Update Workflow

The update process follows a structured human-in-the-loop workflow:

- A rejected operation is detected and logged with full context.
- An agent-based semantic analysis process evaluates whether the rejection represents a legitimate business need or a potential attack probe.
- A policy update proposal is generated, specifying the new operation, its parameters, and its proposed behavioral baseline.
- A human operator reviews and approves or rejects the proposal.
- Upon approval, both SLMs are fine-tuned on the expanded vocabulary and redeployed.

9.4 Human-in-the-Loop Requirement

Fully autonomous capability space updates are explicitly prohibited. This constraint is security-motivated:

If the update mechanism can be triggered autonomously, it becomes an attack surface. An attacker who cannot execute an operation directly may attempt to cause the system to add that operation to its vocabulary through repeated probing. Human approval closes this attack vector.

Human operators retain responsibility for semantic validation, risk assessment of proposed additions, rollback decisions, and policy acceptance. This overhead is bounded: in a stable business environment, vocabulary expansion events are infrequent. The operational cost of human review is low relative to the security guarantee it provides.

10. SECURITY ANALYSIS

10.1 Structurally Prevented Attack Classes

The following attack classes are prevented by the architecture of the Capability Projection Boundary — not by detection logic, but by the absence of representational capacity:

- **Ransomware deployment:** file encryption operations have no semantic vector representation and cannot be projected into or out of the gateway.
- **Arbitrary code execution:** shell and process operations are absent from the vocabulary of both SLMs.
- **SQL and command injection:** injected syntax cannot be mapped to a valid semantic vector.

- **Prompt injection:** even if an attacker embeds override instructions in natural language, the SLM lacks vector representation for concepts outside its training domain.
- **Protocol tunneling:** operations that would establish unauthorized communication channels have no semantic mapping.
- **Outer gateway compromise:** if the outer SLM is manipulated or partially overridden, its output is still constrained to business-domain vectors. The inner SLM cannot receive or process anything beyond this vocabulary.

A common property across all of these prevented attack classes is the mechanism of prevention: the gateway does not reject, flag, or respond to the attack input. It produces no output. This distinction is operationally significant. A system that issues rejection signals provides an attacker with observable feedback: the signal confirms that the input was received, processed, and evaluated. Over sufficient iterations, this feedback enables an adversary to characterize the boundary between permitted and rejected inputs — particularly when attack workflows are automated by AI-capable agents operating at scale. The SCG architecture eliminates this feedback channel. An out-of-domain input produces no capability tag and therefore no downstream response of any kind. From the attacker's perspective, the input is indistinguishable from a dropped network packet. There is no information to extract from the absence of a response, and the probabilistic output characteristics of SLM inference ensure that repeated probing does not yield a stable pattern. The attack surface cannot be mapped because it does not present a learnable signal.

10.2 Residual Risk

The architecture deliberately bounds residual risk to the following categories:

- Fraudulent but capability-valid transactions (e.g., a legitimate purchase order with a manipulated vendor).
- Insider misuse of permitted operations.
- Low-frequency business-logic manipulation that falls within behavioral norms.

These residual risks are characteristic of business process fraud, not infrastructure compromise. They are addressable through existing financial controls, audit procedures, and fraud detection systems — mechanisms that enterprises already operate.

The practical significance is that a successful attacker operating against an SCG-protected system faces the same constraints as a malicious insider: they can misuse legitimate business operations but cannot compromise infrastructure, deploy persistent malware, or exfiltrate data through unauthorized channels.

11. COMPARATIVE ANALYSIS

DIMENSION	CONVENTIONAL ARCHITECTURE	SCG ARCHITECTURE
Security boundary	Network perimeter	Semantic execution space (bidirectional)
Trust model	Authentication implies capability	Authentication does not imply execution rights
Attack surface	All protocol-valid operations	Business-defined vocabulary only
Post-auth breach impact	Broad — attacker inherits session scope	Constrained — capability space limits damage at both layers
Ransomware resistance	Relies on endpoint / EDR	Structural: encrypt concept has no vector representation
Gateway compromise	Single point of failure	Dual-SLM: outer compromise does not bypass inner constraint
Training overhead	N/A	~1 hour initial; minutes for incremental updates
Primary defense unit	Identity / credential	Capability vocabulary

The fundamental shift is from perimeter-centric to capability-centric security. Conventional architectures attempt to control who can access a system. SCG constrains what any accessor — authorized or adversarial — can cause the system to do or disclose.

12. APPLICABILITY AND SCOPE

SCG is particularly suited to systems where:

- The legitimate operational vocabulary is well-defined and relatively stable.
- The consequences of unauthorized execution are severe (financial, regulatory, or operational).
- Post-authentication breach scenarios represent a primary threat model.
- Bidirectional data containment is required — preventing both unauthorized commands inbound and unauthorized data exfiltration outbound.

Target system categories include:

- Enterprise resource planning (ERP) systems.
- Financial transaction processing systems.

- Procurement and supply chain management platforms.
- Customer data access systems subject to regulatory requirements.
- Industrial control system (ICS) interfaces where operational vocabulary is particularly narrow.

SCG is not intended for:

- General-purpose computing environments where the legitimate operational vocabulary cannot be bounded.
- Development environments requiring arbitrary code execution by design.
- Systems where the cost of false rejections exceeds the cost of the attack classes being mitigated.

13. CONCLUSION

This paper introduced the Semantic Constrained Gateway, a post-authentication containment architecture for enterprise core systems.

The SCG model operates from the recognition that modern attack campaigns routinely obtain valid credentials and authenticated sessions. Under this threat model, authentication-centric security architectures provide insufficient protection because they conflate identity verification with execution authorization.

SCG separates these concerns. Authentication remains necessary but no longer implies unrestricted capability. The capability projection boundary — implemented as a pair of domain-specific Small Language Models with intentionally constrained vocabularies — ensures that execution capability is bounded by business-operational semantics regardless of the identity or intent of the requesting entity.

The security guarantee has two structural foundations. First, operations absent from the defined capability space have no projection target — they cannot be capability-tagged, processed, or transmitted through the boundary, not because of a detection rule but because the capability space contains no entry for them. Second, the dual-SLM architecture ensures that even if the outer projector is compromised, its output is still constrained to capability-tagged representations within the business-domain space. The inner projector, receiving only capability-tagged input, applies the same structural constraint at the core system boundary. SCG is not a language gateway. It is a capability projection boundary — and that distinction is the source of its security guarantee.

A further practical advantage is deployment feasibility. Because the target capability space is narrow by design, initial SLM training is achievable in approximately one hour on current TPU hardware. Incremental vocabulary updates require only minutes of fine-tuning. This makes the architecture practical for enterprise adoption and adaptable to evolving business requirements without compromising its core security properties.

Future work should address: empirical evaluation of SLM capability constraint effectiveness against adversarial prompting; formal specification methods for business-domain capability spaces; benchmark studies of training time and inference latency across deployment scales; and the design of the adaptive update mechanism's human review interface to minimize cognitive load while preserving security guarantees.

REFERENCES

[1] Rose, S., Borchert, O., Mitchell, S., & Connelly, S. (2020). Zero Trust Architecture. NIST Special Publication 800-207.

[2] Shostack, A. (2014). Threat Modeling: Designing for Security. Wiley.

[3] Open Policy Agent. https://www.openpolicyagent.org/

[4] Anthropic. (2024). Claude Model Specification. https://anthropic.com/